



TECHNICAL REPORT

Segment Tree

Fast Range Queries and Dynamic Updates

Prepared by

SHAYAN UN NOOR

2305024

MD. SHADMAN SHAHRIYAR SHUVO

2305025

MD. WASIF HAQUE

2305029

Abstract

Segment Trees are hierarchical data structures designed to efficiently process range queries and dynamic updates over an array. A naïve approach may require linear time for every range query or update, which becomes inefficient when the number of operations is large. A Segment Tree addresses this problem by recursively partitioning an array into smaller intervals and storing a summary of each interval at a tree node. This organization allows both point updates and range queries to be performed in $O(\log n)$ time after $O(n)$ preprocessing.

This report presents the structure and construction of a Segment Tree, explains its update and query procedures using worked examples, derives their time complexities, and discusses its generalization to operations such as minimum, maximum, and greatest common divisor. It also introduces lazy propagation for range updates, provides a complete reference implementation, surveys representative applications, and compares Segment Trees with Fenwick Trees, sparse tables, and square-root decomposition.

Keywords: segment tree, range query, point update, associative operation, lazy propagation, Fenwick tree, logarithmic time.

Acknowledgements

The authors would like to thank the course instructors for their guidance throughout this project and for the opportunity to study a practical data structure with a strong theoretical foundation. The worked examples in this report were checked against independent reference computations, and the presentation slides that accompany it share the colour palette used in the figures below.

Contents

Abstract	ii
Acknowledgements	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Motivation	1
1.2 Dynamic Range Query Problem	2
1.3 Objective	2
1.4 Scope and Assumptions	2
1.5 Notation	3
1.6 Organization of the Report	3
2 The Range Query Problem	4
2.1 Problem Definition	4
2.2 Naïve Approach	4
2.3 Limitation of Prefix Precomputation	5
2.4 The Cost Trade-off Visualized	6
2.5 Need for a Dynamic Data Structure	6
3 The Segment Tree	7
3.1 Basic Idea	7
3.2 Structure of a Segment Tree	7
3.3 Interval Representation	8
3.4 Stored Information and Merge Operation	8
3.5 Structural Properties	8
3.6 Array Layout and Index Arithmetic	9
3.6.1 How Much Memory Is Enough?	9
3.7 Invariants	10
3.8 Core Algorithms	10
3.8.1 Build	10
3.8.2 Point Update	11
3.8.3 Range Query	12
3.8.4 Why Range Query Is $O(\log n)$: the Overlap Argument	13
4 Complexity Analysis	15
4.1 Summary of Results	15
4.2 Space Complexity	15
4.3 Why the Bounds Do Not Depend on the Query Pattern	15
5 Worked Example	17
5.1 Setup	17
5.1.1 Build Trace	17
5.2 A Range Query	18
5.3 A Point Update	18

6	Correctness	20
6.1	The Invariant, Restated	20
6.2	Correctness of Build	20
6.3	Correctness of Update	21
6.4	Correctness of Range Query	21
7	Implementation Design and Practical Issues	22
7.1	Array-Based Representation, Revisited	22
7.2	Indexing Conventions	22
7.3	Recursive Versus Iterative Implementations	22
7.4	Numeric Type and Overflow	23
8	Lazy Propagation	24
8.1	The Problem with Naive Range Updates	24
8.2	Lazy Tags	24
8.3	Range Update	24
8.4	Range Query Under Lazy Propagation	25
9	Generalization of Segment Trees	27
9.1	Associative Merge Operations	27
9.2	Range Minimum, Maximum, and GCD	27
9.3	Composite Node Summaries	28
9.4	When Not to Use a Segment Tree	28
10	Applications	29
10.1	Time-Series Data	29
10.2	Leaderboards	29
10.3	Scheduling and Booking	29
10.4	Other Dynamic Range Problems	30
11	Segment Tree vs. Other Data Structures	31
11.1	Naïve Array	31
11.2	Prefix Sum	31
11.3	Square-Root Decomposition	31
11.4	Sparse Table	32
11.5	Fenwick Tree	32
11.6	Comparison Table	33
12	Complexity and Practical Considerations	35
12.1	Choosing a Structure in Practice	35
13	Conclusion	37
A	Reference Pseudocode	38
B	Summary of Complexity	40
	Bibliography	41

List of Figures

2.1	A single point update $A[4] \leftarrow 10$ invalidates five prefix sums (shaded red), $P[4]$ through $P[8]$. In general an update at index i invalidates $n - i + 1$ entries. . .	5
2.2	Cost per operation as a function of n . The linear cost of the two baselines eventually dominates, while the logarithmic Segment Tree cost is almost invisible on this scale.	6
3.1	Heap-style numbering of the Segment Tree for $n = 8$ (top) and the flat array that stores it (bottom). Node k has children $2k$ and $2k + 1$	9
3.2	Query $[1, 4]$ against the array of Figure 3.1. Green nodes are fully covered and returned directly; gold nodes are boundary nodes that recurse further; grey dashed nodes are pruned by the no-overlap case and never visited beyond the check itself. Only $[1, 2]$ and $[3, 4]$ are actually summed.	14

List of Tables

1.1	Summary of notation.	3
2.1	Trade-off between query and update cost across approaches.	6
3.1	Level-by-level partition of $[1, 8]$ in the Segment Tree of the running example. .	7
3.2	Height and memory for several array sizes.	10
3.3	The three cases handled at each recursive step of a range query.	12
4.1	Time and space complexity of the core Segment Tree operations, as proved in Chapter 3.	15
5.1	Node contents after build on $A = [3, 1, 4, 1, 5, 9, 2, 6]$	17
5.2	Recursion trace of query (1,1,8,3,7).	18
5.3	Recursion trace of update (1,1,8,6,7); only the path to leaf 13 is touched. .	18
6.1	Correctness postcondition of each core operation.	20
7.1	Implementation decisions and the reasoning behind the conventional default. .	23
9.1	The same Segment Tree structure, generalized to other associative merge operations.	27
9.2	Design checklist for selecting and implementing a Segment Tree.	28
10.1	Representative applications of the Segment Tree.	29
11.1	Point update and range-sum query implemented on a Fenwick Tree; tree is 1-indexed and initialized to all zeros.	33
11.2	Segment Tree versus the other standard range-query structures.	34
12.1	Summary of Segment Tree time and space complexity.	35
12.2	Practical decision rules for choosing among the structures compared in Chapter 11.	36
B.1	Final complexity reference for every structure discussed in this report.	40

1

Introduction

Many computational systems operate on data that changes continuously, while their users still need aggregate information over contiguous ranges. For example, a system may need the total traded value of instruments 200 through 450, the minimum temperature recorded between 2 pm and 5 pm, or the number of available seats in a given interval. Answering such questions efficiently while the underlying data changes is the central problem studied in this report.

The Segment Tree is a standard data structure for this class of problems. It recursively partitions an array into intervals and stores an aggregate summary for each interval, allowing a broad family of range operations to be supported efficiently.[1, 3] This chapter motivates the problem, states the objective of the report, and outlines how the remaining chapters are organized.

1.1 Motivation

Many computational problems require repeatedly answering queries over a contiguous portion of a dynamically changing dataset. Consider a system tracking n financial instruments, where array A stores their current prices. At 9:31 AM, right after the market opens, 20,000 prices may change every second, while the system must still answer, instantly, “what is the total value of instruments l through r ?” The two fundamental operations required are therefore

$$\text{sum}(l, r) = \sum_{i=l}^r A[i] \tag{1.1}$$

and

$$\text{update}(i, v), \tag{1.2}$$

where $A[i]$ is modified to the value v .

The same pattern appears in very different settings. A multiplayer game must maintain a live leaderboard while thousands of players finish matches. A ticketing platform must find contiguous free seats while other customers are booking. A monitoring dashboard must report the maximum latency of the last five minutes while new measurements stream in. In each case the system is *summarizing a range and then changing part of it*, over and over.

1.2 Dynamic Range Query Problem

The difficulty arises from the interaction between queries and updates. If the array is scanned directly, a query in Equation (1.1) requires $O(n)$ time. If instead a running total is maintained, every update in Equation (1.2) requires $O(n)$ work to keep that total valid. Neither approach scales when both operations occur frequently and at large volume: keeping the sum fresh after every update costs $O(n)$ per update, while keeping updates instantaneous costs $O(n)$ per query. This tension — fast queries versus fast updates — appears in any large system that must *summarize a range and then change part of it*: stock feeds, leaderboards, and analytics dashboards all face it constantly.

Key Idea

Static data can be preprocessed once and queried forever. *Dynamic* data cannot: every preprocessing decision made to speed up queries must be paid for again each time the data changes. The Segment Tree is a principled compromise that keeps both costs logarithmic.

1.3 Objective

This report presents the Segment Tree as a data structure that resolves this tension, achieving $O(\log n)$ time for both operations after an $O(n)$ preprocessing step. We derive its structure, construction, update and query algorithms, and their time complexities; work through a concrete numerical example; generalize the structure to other associative operations; and compare it against alternative approaches. Concretely, the report aims to:

1. define the range-query problem precisely and expose why simple solutions fail (Chapter 2);
2. introduce the Segment Tree, its invariants, and its array layout (Chapter 3);
3. give and analyse algorithms for building, updating, and querying the tree (Sections 3.8.1–3.8.3);
4. show how a single change of the merge operation yields range minimum, maximum, GCD and more (Chapter 9);
5. extend the structure to range updates through lazy propagation (Chapter 8);
6. provide a reference implementation (Chapter 7); and
7. place the structure among its alternatives and applications (Chapters 10–12).

1.4 Scope and Assumptions

Throughout the report we make the following assumptions, unless a section states otherwise.

- The array A has a fixed length $n \geq 1$ and is indexed from 1 to n .
- The merge operation is associative, has an identity element, and can be evaluated in $O(1)$ time.
- Updates replace a single value (*point update*). Range updates are treated separately in Chapter 8.
- Complexities are stated in the word-RAM model, counting one elementary arithmetic or comparison operation as one step.

1.5 Notation

Table 1.1 collects the symbols used throughout the report.

Table 1.1. Summary of notation.

Symbol	Meaning
$A = [a_1, \dots, a_n]$	The underlying array of n elements (1-indexed).
$Q(l, r)$	Range query over the closed interval $[l, r]$.
$\text{update}(i, v)$	Point update: assign $A[i] \leftarrow v$.
$[lo, hi]$	The interval represented by a tree node.
mid	Midpoint $\lfloor (lo + hi)/2 \rfloor$ of a node's interval.
$S(l, r)$	Summary value stored for the interval $[l, r]$.
f, e	Associative merge operation and its identity element.
h	Height of the tree, $h = \lceil \log_2 n \rceil$.
q	Total number of operations (queries plus updates).
$P[i]$	Prefix sum $\sum_{j=1}^i A[j]$.

1.6 Organization of the Report

Chapter 2 formalizes the problem and studies the two baseline solutions. Chapter 3 introduces the Segment Tree and its structural properties. Sections 3.8.1, 3.8.2 and 3.8.3 treat construction, point update, and range query in turn, each with a worked example on the running array

$$A = [2, 1, 5, 3, 7, 4, 6, 8].$$

Chapter 9 generalizes the merge operation, and Chapter 8 adds lazy propagation. Chapter 7 gives a complete implementation. Chapters 10 and 11 discuss applications and alternatives, Chapter 12 consolidates complexity results, and Chapter 13 concludes.

2

The Range Query Problem

Before introducing the Segment Tree it is worth understanding exactly why the problem is hard. This chapter defines the problem formally, analyses the two obvious solutions, and shows that they sit at opposite ends of a trade-off that neither can escape.

2.1 Problem Definition

Let

$$A = [a_1, a_2, \dots, a_n] \quad (2.1)$$

be an array of n elements. A range-sum query is defined as

$$Q(l, r) = \sum_{i=l}^r a_i, \quad 1 \leq l \leq r \leq n, \quad (2.2)$$

and a point update changes a single element,

$$a_i \leftarrow v. \quad (2.3)$$

A workload is a sequence of q operations, each of which is either a query or an update. The goal is to process the entire workload in as little total time as possible, and preferably to bound the cost of *each* operation individually.

Definition 1 (Dynamic range-query problem)

Given an array A of length n , maintain a data structure supporting (i) $Q(l, r)$, returning an aggregate of $A[l..r]$, and (ii) $\text{update}(i, v)$, setting $A[i] \leftarrow v$, such that the two operations may be interleaved in any order.

2.2 Naïve Approach

The most direct way to answer $Q(l, r)$ is to scan the range explicitly:

```
sum(l, r):  
    result = 0  
    for i = l to r:  
        result += A[i]  
    return result
```

For a range of length $r - l + 1$, this costs $O(r - l + 1)$ time, or $O(n)$ in the worst case:

$$T_{\text{query}}(n) = O(n). \quad (2.4)$$

A point update, by contrast, is trivially $O(1)$ under this scheme, since only $A[i]$ is written.

Example 2.1 (Cost of scanning). On $A = [2, 1, 5, 3, 7, 4, 6, 8]$, the query $Q(2, 7)$ reads six elements: $1 + 5 + 3 + 7 + 4 + 6 = 26$. On an array with $n = 10^6$ elements the same query may read up to a million elements, and if 10^5 such queries arrive per second the system must perform 10^{11} element reads per second, which is far beyond what a single core can sustain. $\triangleleft$

2.3 Limitation of Prefix Precomputation

An obvious optimization is to precompute prefix sums,

$$P[i] = \sum_{j=1}^i A[j], \quad (2.5)$$

so that any range sum can be answered in constant time via

$$Q(l, r) = P[r] - P[l - 1]. \quad (2.6)$$

For the running example the prefix array is

$$P = [2, 3, 8, 11, 18, 22, 28, 36],$$

and, for instance, $Q(3, 7) = P[7] - P[2] = 28 - 3 = 25$ on the original array.

This makes queries $O(1)$, but it introduces the opposite problem: whenever a single element $A[i]$ changes, every prefix sum $P[j]$ for $j \geq i$ becomes invalid, so an update costs $O(n)$ to restore correctness. Figure 2.1 shows the damage caused by one update.

	1	2	3	4	5	6	7	8
A before	2	1	5	3	7	4	6	8
P before	2	3	8	11	18	22	28	36
				update(4, 10)				
A after	2	1	5	10	7	4	6	8
P after	2	3	8	18	25	29	35	43

Figure 2.1. A single point update $A[4] \leftarrow 10$ invalidates five prefix sums (shaded red), $P[4]$ through $P[8]$. In general an update at index i invalidates $n - i + 1$ entries.

This gives the fundamental trade-off summarized in Table 2.1.

Table 2.1. Trade-off between query and update cost across approaches.

Method	Query	Point Update
Naïve array	$O(n)$	$O(1)$
Prefix sum	$O(1)$	$O(n)$
Segment Tree	$O(\log n)$	$O(\log n)$

2.4 The Cost Trade-off Visualized

The situation can be seen more clearly by plotting the cost per operation of each approach as n grows. In Figure 2.2 the naïve query cost and the prefix-sum update cost both grow linearly, while the Segment Tree cost for either operation stays close to the horizontal axis.

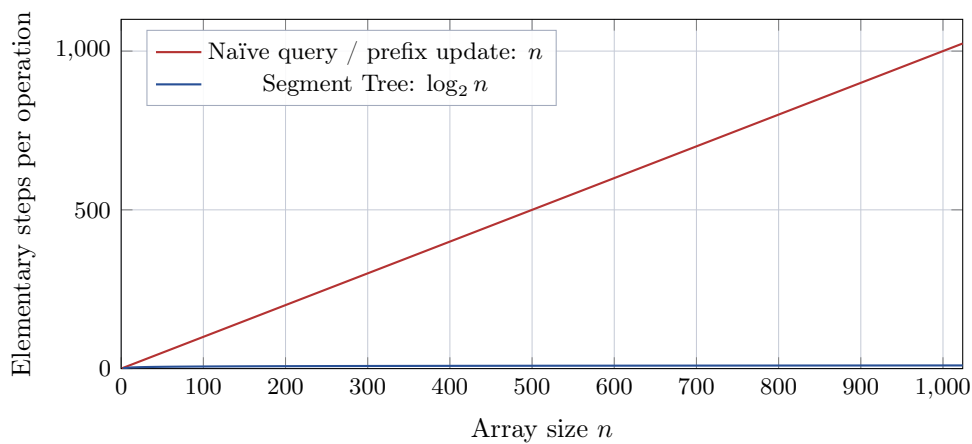


Figure 2.2. Cost per operation as a function of n . The linear cost of the two baselines eventually dominates, while the logarithmic Segment Tree cost is almost invisible on this scale.

2.5 Need for a Dynamic Data Structure

Neither extreme is acceptable when queries and updates are interleaved at scale. What is needed is a structure that balances both costs rather than optimizing one at the expense of the other. The Segment Tree achieves exactly this balance by decomposing the array into a hierarchy of intervals and storing a summary for each one, so that both operations touch only $O(\log n)$ nodes instead of $O(n)$ or $O(1)$ -with-a-hidden- $O(n)$ -cost elsewhere.

Remark

A useful way to think about the trade-off is as a question of *where* the work is done. The naïve array does all its work at query time; the prefix array does all its work at update time. The Segment Tree deliberately spreads the work: each update pays a small amount to keep a hierarchy of partial answers fresh, and each query pays a small amount to combine a handful of those partial answers.

3

The Segment Tree

This chapter introduces the Segment Tree as a data structure: what it stores, how its nodes correspond to intervals, and which structural properties make its operations efficient.

3.1 Basic Idea

A Segment Tree is a binary tree built over an array, where each node represents a contiguous sub-interval of the array and stores a summary value for that interval. The root represents the entire array; each internal node's interval is split into two halves, assigned to its left and right children; and each leaf represents a single array element. Instead of touching the whole array on every operation, the tree recursively splits the problem in half, solves each half, and combines the two answers — and only $O(\log n)$ splits are ever needed to reach any particular piece.

Key Idea

A Segment Tree precomputes the answer to a carefully chosen family of $\approx 2n$ intervals — the *dyadic* intervals obtained by repeated halving — such that every range $[l, r]$ can be written as a disjoint union of $O(\log n)$ of them.

3.2 Structure of a Segment Tree

For an array of size n , the tree has height $h = \lceil \log_2 n \rceil$, and consequently $O(n)$ nodes in total (at most $\sim 2n$ for a complete binary tree representation). Each level of the tree partitions $[1, n]$ into disjoint, contiguous sub-intervals that together cover the whole array exactly once.

Table 3.1 shows how the intervals at each level of the tree for $n = 8$ partition the array.

Table 3.1. Level-by-level partition of $[1, 8]$ in the Segment Tree of the running example.

Level	Intervals at this level	Nodes	Interval length
0	$[1, 8]$	1	8
1	$[1, 4], [5, 8]$	2	4
2	$[1, 2], [3, 4], [5, 6], [7, 8]$	4	2
3	$[1, 1], [2, 2], \dots, [8, 8]$	8	1
Total		15	

3.3 Interval Representation

Each node is identified by the interval $[lo, hi]$ it represents. If $lo = hi$, the node is a leaf corresponding to $A[lo]$. Otherwise, letting

$$mid = \left\lfloor \frac{lo + hi}{2} \right\rfloor, \quad (3.1)$$

the node's left child represents $[lo, mid]$ and its right child represents $[mid + 1, hi]$.

3.4 Stored Information and Merge Operation

For every node representing interval $[l, r]$, define the stored summary as

$$S(l, r) = \sum_{i=l}^r A[i]. \quad (3.2)$$

Given the midpoint $m = \lfloor (l + r)/2 \rfloor$, the summary satisfies the recursive relation

$$S(l, r) = S(l, m) + S(m + 1, r). \quad (3.3)$$

This is the *merge operation*: a parent's summary is obtained by combining its two children's summaries. For sum queries the merge is addition, but as shown in Chapter 9, any associative operation can be substituted in its place.

3.5 Structural Properties

Several simple facts about the tree are used repeatedly in later chapters. We state them here with short proofs.

Lemma 1 (Partition property)

For every level d of the tree, the intervals of the nodes at level d are pairwise disjoint, and their union is contained in $[1, n]$.

Proof. By induction on d . At level 0 there is a single node $[1, n]$. If the claim holds at level d , each node $[lo, hi]$ at that level is replaced at level $d + 1$ by the two disjoint intervals $[lo, mid]$ and $[mid + 1, hi]$ whose union is $[lo, hi]$; nodes with $lo = hi$ have no children. Disjointness and containment are therefore preserved. $\square$

Lemma 2 (Height)

A Segment Tree over $n \geq 1$ elements has height $\lceil \log_2 n \rceil$.

Proof. Each split at least halves the interval length, rounding the larger half up: an interval of length m splits into lengths $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$. After d splits the longest interval therefore has length at most $\lceil n/2^d \rceil$, which equals 1 exactly when $2^d \geq n$, i.e. when $d \geq \lceil \log_2 n \rceil$. $\square$

Lemma 3 (Node count)

A Segment Tree over n elements has exactly $2n - 1$ nodes.

Proof. The tree is a full binary tree (every internal node has exactly two children) with n leaves. In any full binary tree the number of internal nodes is one less than the number of leaves, so the total is $n + (n - 1) = 2n - 1$. $\square$

For $n = 8$ this gives 15 nodes, matching Table 3.1.

3.6 Array Layout and Index Arithmetic

Segment Trees are almost always stored in a flat array rather than with explicit pointers. Numbering the root as node 1, the children of node k are

$$\text{left}(k) = 2k, \quad \text{right}(k) = 2k + 1, \quad \text{parent}(k) = \lfloor k/2 \rfloor. \quad (3.4)$$

This is the same layout used by binary heaps, and it removes the memory overhead of child pointers. Figure 3.1 shows the correspondence for the running example.

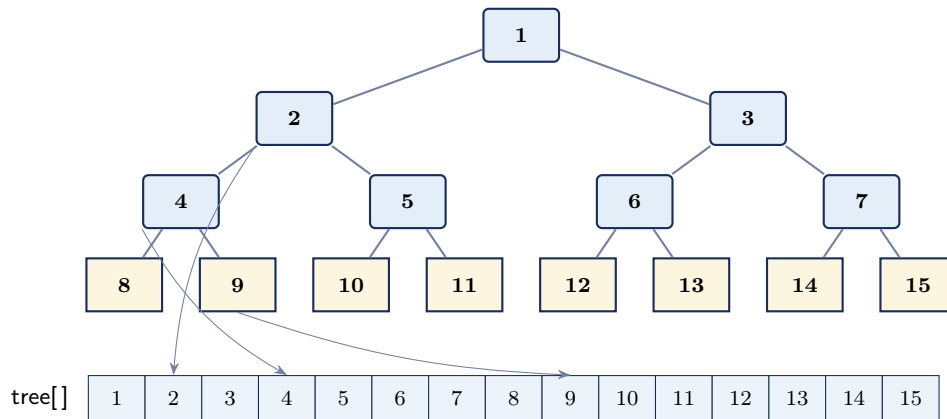


Figure 3.1. Heap-style numbering of the Segment Tree for $n = 8$ (top) and the flat array that stores it (bottom). Node k has children $2k$ and $2k + 1$.

3.6.1 How Much Memory Is Enough?

When n is not a power of two the tree is not perfect, and heap-style indices can reach values larger than $2n - 1$. A safe bound comes from Lemma 3.5: the deepest node has index below 2^{h+1} , where $h = \lceil \log_2 n \rceil$. Since $2^h < 2n$, we get $2^{h+1} < 4n$. This is why implementations conventionally allocate $4n$ cells, as in Table 3.2.

Remark

The true number of nodes is always $2n - 1$ (Lemma 3.5); the $4n$ allocation only pays for the unused gaps that appear in the heap-style numbering when n is not a power of two.

Table 3.2. Height and memory for several array sizes.

n	Height $\lceil \log_2 n \rceil$	Nodes $2n - 1$	Safe bound 2^{h+1}	Rule of thumb $4n$
8	3	15	16	32
100	7	199	256	400
1,000	10	1,999	2,048	4,000
1,024	10	2,047	2,048	4,096
10^6	20	1,999,999	2,097,152	4,000,000
10^9	30	1,999,999,999	2,147,483,648	4,000,000,000

3.7 Invariants

A correct Segment Tree maintains one invariant at all times, which every algorithm in the following chapters must preserve:

$$\text{for every node } v \text{ with interval } [l, r]: \quad \text{tree}[v] = \sum_{i=l}^r A[i]. \quad (3.5)$$

Construction establishes the invariant, queries only read it, and updates must repair it on exactly the nodes where it could have been broken.

3.8 Core Algorithms

Every operation on a Segment Tree is a recursive walk over the same implicit tree described in Section 3.8: at each node the algorithm asks a cheap question about the current interval $[lo, hi]$ and either answers immediately, recurses into one child, or recurses into both. This section develops the three core routines — build, update, and query — one at a time, and Section 5.1 traces each of them through the running $n = 8$ example.

3.8.1 Build

Construction populates every node's stored summary, starting from the leaves and merging upward. Algorithm 3.1 implements it as a single recursive pass.

Listing 3.1. Recursive build.

```

1 void build(int node, int lo, int hi) {
2     if (lo == hi) {
3         tree[node] = A[lo];
4         return;
5     }
6     int mid = (lo + hi) / 2;
7     build(2*node, lo, mid);
8     build(2*node+1, mid+1, hi);
9     tree[node] = tree[2*node] + tree[2*node+1];    // merge
10 }
```

Key Idea

Build never inspects a query range at all — it simply materializes the invariant of Equation (3.5) everywhere, once, so that later queries can trust every node’s stored value without recomputation.

Because each recursive call either reaches a leaf (processing one array element) or splits into exactly two children, the call tree of `build` is precisely the Segment Tree itself: one call per node, $2n - 1$ calls in total by Lemma 3.5. Each call does $O(1)$ work beyond its two recursive calls, so:

Lemma 4 (Build time)

`build` runs in $O(n)$ time and uses $O(n)$ auxiliary space for the `tree` array.

Proof. The number of calls is $2n - 1$ (Lemma 3.5), and each call performs $O(1)$ work outside of its recursive calls, so the total work is $O(2n - 1) = O(n)$. The recursion depth is the tree height $O(\log n)$ (Lemma 3.5), which bounds the auxiliary stack space, while the `tree` array itself occupies $O(n)$ cells. $\square$

3.8.2 Point Update

A point update changes $A[pos]$ to a new value and must restore the invariant of Equation (3.5) at every node whose interval contains pos . Crucially, that set of nodes is not arbitrary: it is exactly the nodes on the root-to-leaf path for pos .

Lemma 5 (Update touches one path)

For a fixed index pos , the set of nodes whose interval contains pos forms a single path from the root to the leaf $[pos, pos]$.

Proof. By Lemma 3.5, the intervals at any one level are pairwise disjoint, so at most one node per level can contain pos . The root’s interval $[1, n]$ always contains pos , and if a node at depth d contains pos then exactly one of its two children’s intervals (by construction, a partition of the parent’s interval) contains pos as well, unless the node is already the leaf $[pos, pos]$. Hence there is exactly one such node at every depth from 0 to $\lceil \log_2 n \rceil$, and consecutive ones are parent and child, i.e. a path. $\square$

This lemma is the entire justification for the update algorithm: descend toward the leaf containing pos , ignoring the sibling subtree at every step, then recompute merges on the way back up.

Listing 3.2. Point update.

```

1 void update(int node, int lo, int hi, int pos, int val) {
2     if (lo == hi) {                               // pos == lo == hi
3         tree[node] = val;
4         return;
5     }
6     int mid = (lo + hi) / 2;
7     if (pos <= mid)
8         update(2*node, lo, mid, pos, val);
9     else

```

```

10     update(2*node+1, mid+1, hi, pos, val);
11     tree[node] = tree[2*node] + tree[2*node+1];    // re-merge on the way back
12 }

```

Common Pitfall

It is tempting to update only the leaf and stop. This leaves every ancestor's stored summary stale, silently corrupting every future query that touches *pos*. The re-merge line at the end of `update` is not optional bookkeeping — it is what restores Equation (3.5).

Lemma 6 (Update time)

`update` runs in $O(\log n)$ time.

Proof. By Lemma 3.8.2, the recursion visits exactly one node per level, and the tree has $\lceil \log_2 n \rceil + 1$ levels by Lemma 3.5. Each visited node performs $O(1)$ work (a comparison, a recursive call, and a merge), so the total work is $O(\log n)$. $\square$

3.8.3 Range Query

A range query for $[ql, qr]$ should return $S(ql, qr) = \sum_{i=ql}^{qr} A[i]$ without touching every element. The algorithm compares the current node's interval $[lo, hi]$ against $[ql, qr]$ and falls into exactly one of three cases at each step.

Table 3.3. The three cases handled at each recursive step of a range query.

Case	Condition	Action
No overlap	$qr < lo$ or $hi < ql$	Return the identity; contributes nothing.
Total overlap	$ql \leq lo$ and $hi \leq qr$	Return <code>tree[node]</code> directly; no need to look deeper.
Partial overlap	otherwise	Recurse into both children and merge their results.

Listing 3.3. Range query.

```

1 int query(int node, int lo, int hi, int ql, int qr) {
2     if (qr < lo || hi < ql)
3         return 0;                                // no overlap (identity for
sum)
4     if (ql <= lo && hi <= qr)
5         return tree[node];                        // total overlap
6     int mid = (lo + hi) / 2;
7     int left = query(2*node, lo, mid, ql, qr);
8     int right = query(2*node+1, mid+1, hi, ql, qr);
9     return left + right;                          // partial overlap
10 }

```

Key Idea

The identity element (here, 0 for sum; $+\infty$ for minimum, $-\infty$ for maximum, 0 for GCD) is what lets the no-overlap case return *something* without a special-cased branch at the call site: merging with the identity never changes the result.

Unlike **build** and **update**, a query does not visit a bounded number of nodes per level in general — it can visit up to four per level, because a query range can cut into an interval from both sides. The next subsection makes this precise, since it is the crux of the $O(\log n)$ query bound and easy to get wrong by assuming the recursion always halves the range.

3.8.4 Why Range Query Is $O(\log n)$: the Overlap Argument

Call a node *fully covered* if it falls into the total-overlap case of Table 3.3, and *boundary* if it falls into the partial-overlap case (its interval straddles ql or qr). No-overlap nodes are pruned immediately and never recurse further, so they contribute no cost beyond the $O(1)$ check that discards them.

Theorem 1 (Query time)

query performs $O(\log n)$ merge operations and visits $O(\log n)$ nodes in total.

Proof. At any fixed level d of the tree, at most two nodes can be boundary nodes: one containing the left endpoint ql and one containing the right endpoint qr (there is at most one node per level containing any single index, by the disjointness half of Lemma 3.5, and a node cannot straddle a query boundary unless it contains ql or qr). A boundary node is exactly the kind of node that recurses into both children, so the number of boundary nodes at level $d+1$ descended from level d 's boundary nodes is at most $2 \cdot 2 = 4$, but by the same argument only at most 2 of those can again be boundary at level $d+1$ — the rest are resolved as fully-covered or no-overlap immediately, which do not recurse further. Formally: define $B(d)$ as the number of boundary nodes at level d . Then $B(0) \leq 1$ and $B(d) \leq 2$ for all $d \geq 1$, because a boundary node has at most two children and at most one of a node's two children can itself be a boundary node on each side — so the left boundary chain and the right boundary chain each advance by at most one boundary node per level. Since a fully-covered node returns immediately without recursing, the total number of nodes visited at any level is $O(1)$, and the recursion depth is $O(\log n)$ by Lemma 3.5. Summing $O(1)$ work over $O(\log n)$ levels gives $O(\log n)$ total. $\square$

Remark

An equivalent, more concrete way to see Theorem 3.8.4: any interval $[ql, qr]$ decomposes into at most $2\lceil \log_2 n \rceil$ dyadic node-intervals, since the decomposition can be built by walking ql and qr upward simultaneously, each accumulating at most one fully-covered node per level. This is the same bound used informally in the Key Idea box of Section 3.

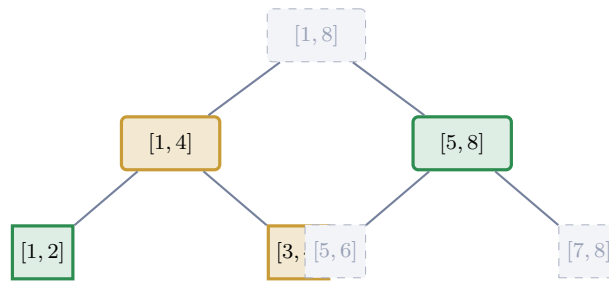


Figure 3.2. Query $[1, 4]$ against the array of Figure 3.1. Green nodes are fully covered and returned directly; gold nodes are boundary nodes that recurse further; grey dashed nodes are pruned by the no-overlap case and never visited beyond the check itself. Only $[1, 2]$ and $[3, 4]$ are actually summed.

4

Complexity Analysis

4.1 Summary of Results

Chapter 3 established the three core algorithms and proved a time bound for each individually. This short chapter collects those results, states the corresponding space bound, and discusses the constants that Table 3.2 already hinted at.

Table 4.1. Time and space complexity of the core Segment Tree operations, as proved in Chapter 3.

Operation	Time	Source
Build	$O(n)$	Lemma 3.8.1
Point update	$O(\log n)$	Lemma 3.8.2
Range query	$O(\log n)$	Theorem 3.8.4

4.2 Space Complexity

The `tree` array is indexed with the heap-style scheme of Equation (3.4), so its size must accommodate the largest index any node can receive rather than merely the true node count of $2n - 1$ (Lemma 3.5).

Lemma 1 (Space bound)

Allocating $O(n)$ cells — specifically $4n$, per Table 3.2 — suffices for the `tree` array regardless of whether n is a power of two.

Proof. By Lemma 3.5 the tree has height $h = \lceil \log_2 n \rceil$, so every node index under the numbering of Equation (3.4) is strictly below 2^{h+1} . Since $2^h < 2n$ (because $h = \lceil \log_2 n \rceil$ implies $2^{h-1} < n \leq 2^h$), it follows that $2^{h+1} < 4n$, so $4n$ cells is a safe — if slightly generous — upper bound. $\square$

4.3 Why the Bounds Do Not Depend on the Query Pattern

A recurring point of confusion is whether an adversarial sequence of queries and updates could push the cost above $O(\log n)$ per operation. It cannot: Lemma 3.8.2 shows the update path is determined purely by *pos*, and Theorem 3.8.4’s boundary-node argument bounds the number of visited nodes purely by the tree’s height, independent of which values are stored or which merge results are equal to one another. The bounds are therefore *worst-case per operation*, not amortized, which is one of the practical advantages of a Segment Tree over structures that only offer amortized guarantees.

Remark

This worst-case, per-operation guarantee is also why Segment Trees are preferred in settings with hard latency requirements, such as the financial-instrument scenario of Section 1.1: an amortized bound could in principle let a single unlucky query take $O(n)$ time, which a Segment Tree never does.

5

Worked Example

This chapter traces build, several updates, and several queries through one concrete array, so that the abstract algorithms of Chapter 3 can be checked step by step against explicit numbers.

5.1 Setup

Let $n = 8$ and

$$A = [3, 1, 4, 1, 5, 9, 2, 6],$$

the same array underlying Figure 3.1 and Figure 3.1, indexed $A[1], \dots, A[8]$.

5.1.1 Build Trace

Table 5.1 lists the stored sum at every node after `build(1,1,8)` completes, using the heap-style indices of Equation (3.4).

Table 5.1. Node contents after `build` on $A = [3, 1, 4, 1, 5, 9, 2, 6]$.

Node	Interval	Sum	Level
1	[1, 8]	31	0
2	[1, 4]	9	1
3	[5, 8]	22	1
4	[1, 2]	4	2
5	[3, 4]	5	2
6	[5, 6]	14	2
7	[7, 8]	8	2
8	[1, 1]	3	3
9	[2, 2]	1	3
10	[3, 3]	4	3
11	[4, 4]	1	3
12	[5, 5]	5	3
13	[6, 6]	9	3
14	[7, 7]	2	3
15	[8, 8]	6	3

Each leaf (nodes 8–15) is filled directly from A ; each internal node is then the sum of its two children, matching Equation (3.3). The root value 31 is the total of all eight elements, as expected.

5.2 A Range Query

Consider $\text{query}(3, 7)$. Table 5.2 follows the recursion of Algorithm 3.3 call by call.

Table 5.2. Recursion trace of $\text{query}(1, 1, 8, 3, 7)$.

Node	Interval	Case	Action
1	[1, 8]	Partial	Recurse into nodes 2 and 3.
2	[1, 4]	Partial	Recurse into nodes 4 and 5.
3	[5, 8]	Partial	Recurse into nodes 6 and 7.
4	[1, 2]	No overlap ($hi = 2 < ql = 3$)	Return 0.
5	[3, 4]	Total overlap	Return $\text{tree}[5] = 5$.
6	[5, 6]	Total overlap	Return $\text{tree}[6] = 14$.
7	[7, 8]	Partial	Recurse into nodes 14 and 15.
14	[7, 7]	Total overlap	Return $\text{tree}[14] = 2$.
15	[8, 8]	No overlap ($lo = 8 > qr = 7$)	Return 0.

Combining the returned values bottom-up: node 7 returns $2+0 = 2$; node 3 returns $14+2 = 16$; node 2 returns $0+5 = 5$; node 1 returns $5+16 = 21$. Indeed $A[3] + A[4] + A[5] + A[6] + A[7] = 4 + 1 + 5 + 9 + 2 = 21$, confirming the trace. Only 9 of the 15 nodes were visited, and only two nodes (5 and 6) contributed a value directly without further recursion — exactly the fully-covered decomposition described in the remark after Theorem 3.8.4.

5.3 A Point Update

Now apply $\text{update}(6, 7)$, changing $A[6]$ from 9 to 7. By Lemma 3.8.2 this touches exactly the path from the root to leaf $[6, 6]$, i.e. nodes $1 \rightarrow 3 \rightarrow 6 \rightarrow 13$.

Table 5.3. Recursion trace of $\text{update}(1, 1, 8, 6, 7)$; only the path to leaf 13 is touched.

Node	Interval	Effect
13	[6, 6]	Base case; $\text{tree}[13] \leftarrow 7$.
6	[5, 6]	Re-merge: $\text{tree}[6] \leftarrow \text{tree}[12] + \text{tree}[13] = 5 + 7 = 12$.
3	[5, 8]	Re-merge: $\text{tree}[3] \leftarrow \text{tree}[6] + \text{tree}[7] = 12 + 8 = 20$.
1	[1, 8]	Re-merge: $\text{tree}[1] \leftarrow \text{tree}[2] + \text{tree}[3] = 9 + 20 = 29$.

Nodes 2, 4, 5, 7 and every leaf other than 13 are untouched, since none of their intervals contain index 6 — exactly as Lemma 3.8.2 predicts. Re-running $\text{query}(3, 7)$ after this update would now correctly return $4 + 1 + 5 + 7 + 2 = 19$, reflecting the updated value without any node outside the single path having been revisited.

Remark

Table 5.3 and Table 5.2 together illustrate the asymmetry between the two operations: an update always touches a fixed-size path of $O(\log n)$ nodes regardless of which index

changed, while a query's cost depends on how the range $[ql, qr]$ happens to align with the dyadic intervals of the tree — though it is $O(\log n)$ in every case, by Theorem 3.8.4.

6

Correctness

Chapter 4 established *how fast* the three core algorithms run; this short chapter states *why they are right*, tying each argument back to the invariant of Equation (3.5) and the lemmas already proved in Chapter 3.

6.1 The Invariant, Restated

Every correctness argument in this report reduces to one statement, repeated here for convenience:

$$\text{for every node } v \text{ with interval } [l, r]: \quad \text{tree}[v] = S(l, r) = \sum_{i=l}^r A[i]. \quad (3.5 \text{ revisited})$$

build establishes it, **query** only reads it, and **update** must restore it after exactly one leaf changes.

Table 6.1. Correctness postcondition of each core operation.

Operation	Postcondition
Build	Every node satisfies Equation (3.5) for its own interval.
Point update	Every node on the root-to-leaf path for <i>pos</i> is recomputed from already-correct children; every other node is untouched and remains correct because its interval does not contain <i>pos</i> .
Range query	The set of nodes returned directly (the fully-covered case) is pairwise disjoint and its union is exactly $[ql, qr]$; merging them therefore gives $S(ql, qr)$.

6.2 Correctness of Build

Theorem 1 (Build correctness)

After **build**(1,1,n) completes, Equation (3.5) holds at every node.

Proof. By induction on subtree height. A leaf $[lo, lo]$ is assigned $\text{tree}[\text{node}] = A[lo] = S(lo, lo)$ directly, the base case. For an internal node $[lo, hi]$ with $mid = \lfloor (lo + hi)/2 \rfloor$, the two recursive calls establish the invariant at the children $[lo, mid]$ and $[mid+1, hi]$ by the inductive hypothesis, and the subsequent merge sets $\text{tree}[\text{node}] = \text{tree}[2 \cdot \text{node}] + \text{tree}[2 \cdot \text{node} + 1] = S(lo, mid) + S(mid + 1, hi) = S(lo, hi)$ by Equation (3.3). $\square$

6.3 Correctness of Update

Theorem 2 (Update correctness)

If Equation (3.5) holds before `update(1,1,n,pos,val)` and $A[pos]$ is conceptually set to `val`, then Equation (3.5) again holds at every node after the call returns.

Proof. By Lemma 3.8.2, the nodes containing pos form exactly the root-to-leaf path visited by the recursion, and only these nodes have their interval's sum affected by the change to $A[pos]$; every node off this path has an interval that excludes pos , so its correct value $S(l, r)$ is unaffected by the change and, since the algorithm never modifies it, it remains correct. Along the path, the leaf is set directly to $val = S(pos, pos)$, and each ancestor is recomputed as the merge of its two children on the way back up the recursion; since both children are correct at the moment of merging (the deeper one by having just been fixed, the shallower sibling by not having changed), Equation (3.3) guarantees the parent becomes correct as well. Induction up the path completes the argument. $\square$

6.4 Correctness of Range Query

Theorem 3 (Query correctness)

Assuming Equation (3.5) holds throughout, `query(1,1,n,ql,qv)` returns $S(ql, qr)$.

Proof. Every value returned by the recursion is one of: the identity (no-overlap case), a stored $tree[node] = S(lo, hi)$ for a node fully contained in $[ql, qr]$ (total-overlap case), or a merge of two such recursively-correct values (partial-overlap case). The no-overlap contributions vanish under the merge since the identity leaves a merge unchanged. What remains is a sum of $S(l_i, r_i)$ over the fully-covered nodes reached by the recursion. These intervals are pairwise disjoint by Lemma 3.5 restricted to the nodes actually visited, and their union is exactly $[ql, qr]$: every index in $[ql, qr]$ eventually lands inside some node that becomes fully covered, because the recursion only stops descending once a node's interval is entirely inside $[ql, qr]$ or entirely outside it, and it always reaches a leaf if neither holds. By additivity of S over a partition of $[ql, qr]$, the sum of the disjoint pieces equals $S(ql, qr)$. $\square$

Remark

Theorem 6.4's disjoint-cover argument is exactly what Figure 3.2 illustrates concretely: the two green nodes $[1, 2]$ and $[3, 4]$ are disjoint and their union is $[1, 4]$, the requested range.

7

Implementation Design and Practical Issues

The preceding chapters fix the algorithmic content of a Segment Tree; this chapter addresses the engineering decisions that turn that content into working code.

7.1 Array-Based Representation, Revisited

Section 3.8.1 already introduced the heap-style array layout of Equation (3.4) and its $4n$ allocation rule (Lemma 4.2). This remains the default choice: it avoids pointer objects entirely, keeps sibling and parent access to simple arithmetic, and gives predictable, cache-friendly memory access compared to a pointer-based tree.

7.2 Indexing Conventions

This report’s mathematical exposition uses one-indexed intervals such as $[1, n]$ throughout, matching the code listings. Most production languages use zero-based arrays, in which case the interval becomes $[0, n - 1]$ and every occurrence of $mid = \lfloor (lo + hi)/2 \rfloor$, the leaf test $lo = hi$, and the child ranges $[lo, mid]/[mid + 1, hi]$ carry over unchanged in form — only the array’s own bounds shift. Mixing the two conventions within a single implementation, rather than choosing one and committing to it, is the most common source of off-by-one bugs in Segment Tree code.

Common Pitfall

A particularly common variant of this bug is querying with an inclusive-exclusive range $[ql, qr)$ against a tree built with inclusive-inclusive intervals $[lo, hi]$. The no-overlap and total-overlap conditions in Table 3.3 are only correct for two intervals using the *same* convention.

7.3 Recursive Versus Iterative Implementations

The recursive form used throughout Algorithms 3.1–3.3 mirrors the mathematical definitions directly and is the easiest to verify against the proofs of Chapter 6. An iterative, bottom-up formulation over the same heap-style array avoids function-call overhead and is common in performance-sensitive competitive-programming code, at the cost of being noticeably harder to adapt once lazy propagation (Chapter 8) is introduced. Both forms implement the identical interval decomposition; the choice is an engineering trade-off, not a difference in asymptotic behavior.

7.4 Numeric Type and Overflow

For a sum Segment Tree, the stored aggregate at the root can be up to n times larger than any individual element. A 32-bit integer type that comfortably holds each $A[i]$ can silently overflow when summing n of them; 64-bit integers are the conventional safe default for sum aggregates over anything but the smallest inputs. This concern does not apply in the same way to min, max, or GCD trees (Chapter 9), whose stored values never exceed the range of the input elements themselves.

Table 7.1. Implementation decisions and the reasoning behind the conventional default.

Concern	Typical choice	Reason
Node storage	Flat array	Low overhead, predictable memory access (Section 7).
Tree allocation	$4n$ cells	Safe bound under heap-style indexing (Lemma 4.2).
Numeric type (sum)	64-bit integer	Root aggregate can be $n \times$ larger than any element.
Indexing	One consistent convention	Prevents boundary/midpoint mismatches (Section 7.2).
Recursion depth	$O(\log n)$	Safe for essentially all practical n (Lemma 3.5).

8

Lazy Propagation

8.1 The Problem with Naive Range Updates

Every algorithm so far modifies a single array position per update. A *range update* — for example, “add 5 to every element in $[3, 7]$ ” — has no efficient naive implementation on a Segment Tree: applying Algorithm 3.2 once per index in the range costs $O(r - l + 1)$ point updates, each $O(\log n)$, for a total of $O((r - l + 1) \log n)$, which degrades to $O(n \log n)$ for a full-array update. This defeats the purpose of the data structure.

Key Idea

Lazy propagation defers work: instead of eagerly pushing a range update down to every affected leaf, a node records a pending change and applies it to its *own* stored aggregate immediately, but only pushes the pending change further down to its children the next time that subtree is actually visited[3].

8.2 Lazy Tags

Each node gains a second array, conventionally called **lazy**, storing a pending increment not yet applied to the node’s children. A node’s own **tree** value is always kept up to date — it already reflects its own pending tag — but its children’s **tree** values may be stale until the tag is *pushed down*.

Listing 8.1. Pushing a pending tag down to both children before recursing into either one.

```
1 void pushDown(int node, int lo, int hi) {
2     if (lazy[node] == 0) return;
3     int mid = (lo + hi) / 2;
4     int lLen = mid - lo + 1, rLen = hi - mid;
5
6     tree[2*node]      += lazy[node] * lLen;
7     lazy[2*node]      += lazy[node];
8     tree[2*node+1]    += lazy[node] * rLen;
9     lazy[2*node+1]    += lazy[node];
10
11     lazy[node] = 0;
12 }
```

// fully delegated

8.3 Range Update

A range update applies the same three-way case split as **query** (Table 3.3): a node fully inside the update range absorbs the change directly and tags its children lazily instead of recursing;

a node with no overlap is pruned; a partially overlapping node must first push down any tag of its own, then recurse into both children, then re-merge.

Listing 8.2. Range update with lazy propagation (add val to every element in the update interval).

```

1 void rangeUpdate(int node, int lo, int hi, int ul, int ur, int val) {
2     if (ur < lo || hi < ul) return;           // no overlap
3     if (ul <= lo && hi <= ur) {               // total overlap
4         tree[node] += val * (hi - lo + 1);
5         lazy[node] += val;
6         return;
7     }
8     pushDown(node, lo, hi);                   // partial overlap
9     int mid = (lo + hi) / 2;
10    rangeUpdate(2*node, lo, mid, ul, ur, val);
11    rangeUpdate(2*node+1, mid+1, hi, ul, ur, val);
12    tree[node] = tree[2*node] + tree[2*node+1];
13 }

```

8.4 Range Query Under Lazy Propagation

The query algorithm (Algorithm 3.3) needs exactly one addition: a call to `pushDown` before recursing into a partially overlapping node, so that stale children are corrected before their values are read.

Listing 8.3. Range query, adapted for a tree with lazy tags.

```

1 int query(int node, int lo, int hi, int ql, int qr) {
2     if (qr < lo || hi < ql) return 0;
3     if (ql <= lo && hi <= qr) return tree[node];
4     pushDown(node, lo, hi);                   // only new line vs. Algorithm 3
5     int mid = (lo + hi) / 2;
6     int left = query(2*node, lo, mid, ql, qr);
7     int right = query(2*node+1, mid+1, hi, ql, qr);
8     return left + right;
9 }

```

Lemma 1 (Lazy operation time)

With lazy propagation, both `rangeUpdate` and `query` run in $O(\log n)$ time.

Proof. The recursion structure of `rangeUpdate` is identical to that of Algorithm 3.3 (Table 3.3 governs both), so the same overlap argument used in Theorem 3.8.4 bounds the number of visited nodes by $O(\log n)$. Each visited node now also performs a `pushDown`, which is $O(1)$ work, so the total remains $O(\log n)$. The query analysis is unchanged from Theorem 3.8.4 except for the same $O(1)$ -per-node `pushDown` addition. $\square$

Common Pitfall

Omitting the `pushDown` call inside `query` (or inside `rangeUpdate`'s partial-overlap branch) is the single most common bug in lazy Segment Tree implementations: the parent's `tree` value is correct, but a child read directly — without first being corrected — may still

reflect a stale value from before a pending tag was applied.

9

Generalization of Segment Trees

9.1 Associative Merge Operations

One of the most useful properties of the Segment Tree is that its *shape* — the recursive interval decomposition proved correct in Chapter 6 — never changes when the underlying problem changes; only the stored summary and the merge rule f need to be redefined. For the recursive query implementation used in this report, the merge operation f must be associative and have an identity element e so that the no-overlap case can return a neutral value. The associativity requirement is

$$f(f(a, b), c) = f(a, f(b, c)) \quad \text{for all } a, b, c. \quad (9.1)$$

Associativity is what allows the summaries of disjoint child intervals to be merged without depending on the order in which the interval decomposition is formed. The identity element e , with $f(x, e) = x$, is additionally needed by the particular query routine used here to handle the no-overlap case of Table 3.3.

Table 9.1. The same Segment Tree structure, generalized to other associative merge operations.

Problem	Node stores	Merge rule	Identity
Range sum	Sum	$a + b$	0
Range minimum	Minimum	$\min(a, b)$	$+\infty$
Range maximum	Maximum	$\max(a, b)$	$-\infty$
Range GCD	GCD	$\gcd(a, b)$	0
Any associative rule	Any summary	swap in the rule	swap in the identity

9.2 Range Minimum, Maximum, and GCD

For each row of Table 9.1, Algorithms 3.1–3.3 are *structurally unchanged* — only the merge line (the single addition in each listing) is replaced, e.g. `tree[node] = min(tree[2*node], tree[2*node+1])` for range minimum. Because Theorem 6.2 through Theorem 6.4 and the complexity bounds of Chapter 4 used nothing about $+$ beyond associativity and the identity property, all of them — $O(n)$ build, $O(\log n)$ update, $O(\log n)$ query — carry over unchanged to every row of the table, and to any other associative operation not listed.

Remark

Range minimum and maximum are not, in general, invertible the way sum is: there is no operation g with $g(\min(a, b), a) = b$ analogous to subtraction undoing addition. This is why a *point* update (Algorithm 3.2) works unchanged for min/max trees — it recomputes ancestors from children rather than trying to “undo” a stale value — while certain range-update tricks that exploit invertibility for sum trees do not transfer to min/max without

modification.

9.3 Composite Node Summaries

A node need not store a single scalar. It may store a small structure of several jointly-merged statistics, provided the combined structure itself satisfies Equation (9.1). A classic example is the *maximum-subarray-sum* query: each node stores its interval's total sum, best prefix sum, best suffix sum, and best subarray sum, and the merge rule combines two children's four-tuples into a parent four-tuple in $O(1)$ time. This turns the Segment Tree from a range-aggregate container into a general framework for a much larger class of interval algorithms.

9.4 When Not to Use a Segment Tree

A Segment Tree is not automatically the best choice for every range problem, and the design checklist below is meant to be worked through before committing to one.

Table 9.2. Design checklist for selecting and implementing a Segment Tree.

Question	Design implication
What is queried?	Defines the node summary and merge operation f .
What is updated?	Point updates only, or ranges? The latter needs lazy propagation (Chapter 8).
Is the merge associative?	Required for Table 3.3's decomposition to be well defined at all.
Is an identity required?	Needed so the no-overlap case (Table 3.3) has something neutral to return.
Are there any updates at all?	If not, a plain prefix-sum array answers range sums in $O(1)$ with no tree.
Is the operation just addition with point updates?	A Fenwick Tree (Chapter 11) gives the same $O(\log n)$ bounds with less memory and a simpler implementation.

10

Applications

Table 10.1 surveys representative domains where the Segment Tree’s core pattern — *summarize a range, then change part of it, repeatedly and at scale* — arises naturally, extending the financial-instrument scenario first introduced in Section 1.1.

Table 10.1. Representative applications of the Segment Tree.

Application	Dynamic data	Possible query
Financial time series	Prices over time	Sum/min/max over a time interval
Leaderboards	Player scores	Range statistics; augmented for rank/ k -th order
Scheduling & booking	Time-slot availability	Aggregate availability; augmented for longest run
Analytics dashboards	Continuously changing measurements	Range minimum/maximum/-sum

10.1 Time-Series Data

In a system tracking financial instrument prices — the running motivating example of Section 1.1 — a plain sum Segment Tree already supports instant range-sum, range-min, or range-max queries over any contiguous time window, even as thousands of price updates arrive per second, at the worst-case $O(\log n)$ per operation guaranteed by Theorem 3.8.4 and Lemma 3.8.2.

10.2 Leaderboards

The plain sum tree described up to this point does not, by itself, answer rank or k -th-largest queries. However, per the generalization principle of Chapter 9, the same tree shape can be *augmented* to store score-frequency counts at each node rather than a running sum, which allows rank queries and k -th-order-statistic lookups to be answered in $O(\log n)$, with score updates handled just as fast.

10.3 Scheduling and Booking

Similarly, locating or reserving a free contiguous range of seats or time slots requires an augmented Segment Tree in which each node stores additional composite information — such as the count of free slots together with the longest prefix, suffix, and interior run of free

slots within its interval, in the spirit of Section 9.3's maximum-subarray example — so that availability queries and reservations can still be resolved in $O(\log n)$.

10.4 Other Dynamic Range Problems

More generally, any system that must repeatedly answer “what is true across this range?” while “something inside the range keeps changing” is a candidate for a Segment Tree, regardless of the specific domain, provided a suitable associative merge (Section 9.1) can be found for whatever “true across this range” means for that problem.

11

Segment Tree vs. Other Data Structures

The preceding chapters developed the Segment Tree in isolation. This chapter places it alongside the other standard tools for range queries, so that choosing among them becomes a matter of matching a structure's guarantees to a problem's requirements rather than defaulting to the most recently learned technique.

11.1 Naïve Array

A plain array supports a point update in $O(1)$ by simply overwriting $A[i]$, but a range query has no shortcut: every one of the $r - l + 1$ elements in $[l, r]$ must be read, giving $O(n)$ in the worst case. This provides the baseline against which the logarithmic Segment Tree bounds can be compared.

11.2 Prefix Sum

Precomputing $P[i] = \sum_{k=1}^i A[k]$ once, in $O(n)$, answers any range-sum query as $P[r] - P[l-1]$ in $O(1)$ — unbeatable when the array never changes. The weakness is symmetric to the naïve array's: a single point update at position i invalidates $P[i], P[i+1], \dots, P[n]$, forcing an $O(n)$ rebuild of the suffix of the prefix array. Table 2.1 already used this trade-off to motivate the Segment Tree in Chapter 2; it is restated here as the first row of Table 11.2 for completeness.

Common Pitfall

Prefix sums do not provide the same direct $O(1)$ range-query formula for arbitrary range minimum or maximum queries. In particular, two prefix minima cannot generally be combined by subtraction or another inverse operation to recover $\min(A[l], \dots, A[r])$. Thus prefix sums are especially useful for static range-sum and other settings where an appropriate inverse operation exists.

11.3 Square-Root Decomposition

Square-root decomposition splits $A[1 \dots n]$ into $\approx \sqrt{n}$ contiguous blocks of size $\approx \sqrt{n}$ each, storing one aggregate per block. A point update recomputes only the one block containing the changed index, in $O(\sqrt{n})$. A range query $[l, r]$ reads whole blocks fully contained in $[l, r]$ directly from their precomputed aggregates, and falls back to scanning the at most two partial blocks at the ends element by element, for a total of $O(\sqrt{n})$.

Key Idea

Square-root decomposition is best understood as sitting exactly between the two extremes of Sections 11.2–above: it balances update and query cost instead of making one of them $O(1)$ at the other’s expense, in the same spirit as the Segment Tree, but with a shallower two-level hierarchy (blocks of elements) instead of a full $O(\log n)$ -deep binary decomposition.

Because both of its operations are $O(\sqrt{n})$ rather than $O(\log n)$, square-root decomposition is asymptotically slower than a Segment Tree for large n , but its block structure is simple to implement and adapts readily to more exotic per-block bookkeeping (for example, a per-block “lazy” add, mirroring Chapter 8 at a coarser granularity), which is why it remains popular as a quick, low-risk substitute when a Segment Tree’s implementation cost is not justified.

11.4 Sparse Table

A sparse table precomputes, for every position i and every power of two $2^k \leq n$, the aggregate of the block $[i, i + 2^k - 1]$, using $O(n \log n)$ time and space overall[6]. For an *idempotent* merge operation — one satisfying $f(a, a) = a$, such as min, max, or bitwise AND/OR — any range query $[l, r]$ can then be answered in $O(1)$ by covering it with two (possibly overlapping) precomputed blocks of size $2^{\lfloor \log_2(r-l+1) \rfloor}$ and merging them, since overlap causes no double-counting under idempotence.

Common Pitfall

The sparse table’s $O(1)$ query is available *only* for idempotent operations and *only* on a static array: unlike the overlap in min/max queries, overlap under sum double-counts the shared region, so a sparse table cannot answer range-sum queries this way; and because every precomputed block would need to be recomputed, it does not support updates at all in its basic form. It is best reserved for static range-minimum/maximum queries (for instance, as a building block for the Lowest Common Ancestor problem), not as a general-purpose competitor to the Segment Tree.

11.5 Fenwick Tree

A Fenwick Tree (Binary Indexed Tree) supports both point update and range-sum query in $O(\log n)$, matching the Segment Tree’s asymptotic bounds for this specific combination, but with a smaller memory footprint[4, 5] (an array of size $n+1$, versus the $4n$ bound of Lemma 4.2) and a markedly simpler implementation: each of its two core operations is a short loop that repeatedly adds or subtracts the lowest set bit of an index, with no explicit tree, recursion, or merge function to write.

Table 11.1. Point update and range-sum query implemented on a Fenwick Tree; `tree` is 1-indexed and initialized to all zeros.

```

1 void update(int i, int delta) {
2     for (; i <= n; i += i & (-i))
3         tree[i] += delta;
4 }
5
6 int prefixSum(int i) {
7     int s = 0;
8     for (; i > 0; i -= i & (-i))
9         s += tree[i];
10    return s;
11 }
12
13 int rangeSum(int l, int r) {
14     return prefixSum(r) - prefixSum(l - 1);
15 }

```

The Fenwick Tree’s simplicity comes at a cost of generality. It is naturally suited only to operations that, like sum, admit an inverse (so that a range query can be expressed as a difference of two prefix queries, exactly as in Section 11.2); min and max have no such inverse, so a Fenwick Tree cannot answer arbitrary range-minimum or range-maximum queries the way a Segment Tree can via Chapter 9. Range *updates* are also not its basic operation: a Fenwick Tree can be adapted, with a second auxiliary array, to support range-add/point-query or even range-add/range-sum-query, but this adaptation is closer in spirit to the lazy propagation of Chapter 8 bolted onto an unrelated structure than to a native feature of the Fenwick Tree itself.

11.6 Comparison Table

Table 11.2 consolidates every structure discussed above. Rather than declaring one structure to be strictly superior, the appropriate conclusion is that each occupies a distinct point in the trade-off between update cost, query cost, memory, and the class of operations it supports — exactly the axes the design checklist of Table 9.2 already asks the reader to consider.

Table 11.2. Segment Tree versus the other standard range-query structures.

Structure	Update	Query	Build	Memory	Supports
Naïve array	$O(1)$	$O(n)$	–	$\sim n$	Any operation; queries require scanning the range.
Prefix sum	$O(n)$	$O(1)$	$O(n)$	$\sim n$	Invertible operations such as sum; mainly static workloads.
Sqrt decomposition	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(n)$	$\sim n$	Any associative operation; simple to extend.
Sparse table	unsupported	$O(1)$	$O(n \log n)$	$\sim n \log n$	Idempotent operations such as min/max on static data.
Fenwick tree	$O(\log n)$	$O(\log n)$	$O(n)$	$\sim n$	Invertible operations such as sum; simpler and less flexible.
Segment tree	$O(\log n)$	$O(\log n)$	$O(n)$	$\sim 4n$	Broad class of associative aggregates; suitable range updates via Ch. 8.

Remark

Among the structures compared here, the Segment Tree provides a particularly flexible combination of logarithmic point updates, logarithmic range queries, and support for a broad class of associative aggregates. With lazy propagation, it can also support suitable range-update operations. Other structures may be preferable when a problem has more specialized requirements, which is why the design checklist of Table 9.2 is useful.

12

Complexity and Practical Considerations

Table 12.1 consolidates the complexity results derived across Chapters 4–6 and extended to range updates in Chapter 8.

Table 12.1. Summary of Segment Tree time and space complexity.

Operation	Time Complexity	Explanation
Build	$O(n)$	Every tree node is constructed exactly once (Lemma 3.8.1).
Point update	$O(\log n)$	Exactly one root-to-leaf path is re-computed (Lemma 3.8.2).
Range query	$O(\log n)$	At most $O(1)$ partially-overlapping nodes are expanded per level (Theorem 3.8.4).
Range update	$O(\log n)$	Same overlap argument as range query, plus $O(1)$ push-down per node (Lemma 8.4).
Space	$O(n)$	The tree occupies at most $4n$ array cells (Lemma 4.2).

$$\begin{array}{l}
 \text{Build} = O(n) \\
 \text{Point update} = O(\log n) \\
 \text{Range query} = O(\log n) \\
 \text{Range update (lazy)} = O(\log n) \\
 \text{Space} = O(n)
 \end{array} \tag{12.1}$$

12.1 Choosing a Structure in Practice

Bringing together Table 11.2 and Table 9.2, three practical rules of thumb summarize this report’s guidance:

Table 12.2. Practical decision rules for choosing among the structures compared in Chapter 11.

If...	... then consider
The array never changes	A prefix sum array (sum) or a sparse table (min/max); both give $O(1)$ queries with no per-query tree traversal.
Only point updates and sums are needed	A Fenwick Tree, for its smaller memory footprint and simpler implementation.
Range updates, or a non-invertible merge (min, max, GCD, or a composite statistic as in Section 9.3), are needed	A Segment Tree, optionally with lazy propagation (Chapter 8).
Implementation time is scarce and $O(\sqrt{n})$ is acceptable	Square-root decomposition, as a lower-risk fallback.

13

Conclusion

The Segment Tree resolves the fundamental tension between fast range queries and fast point updates by recursively decomposing an array into a hierarchy of intervals, each summarized at a tree node (Chapter 3). This organization allows both operations to be performed in $O(\log n)$ time after an $O(n)$ build, as established by the overlap argument for queries (Theorem 3.8.4) and the single-path argument for updates (Lemma 3.8.2), and proved formally correct in Chapter 6.

With an associative merge operation and the identity element required by the query routine, the same tree structure can be adapted to range minimum, maximum, GCD, and a wide class of composite statistics (Chapter 9). It can also be extended to suitable range-update operations through lazy propagation (Chapter 8) while retaining logarithmic worst-case complexity per operation (Lemma 8.4). This combination of generality and efficiency makes Segment Trees useful across domains such as financial time series, live leaderboards, and scheduling systems (Chapter 10).

Chapter 11 situated the Segment Tree among other standard range-query structures. It trades some memory and implementation simplicity, relative to a Fenwick Tree, a prefix-sum array, or a sparse table, for greater flexibility in the class of aggregates and updates it can support. As Table 12.2 makes concrete, the appropriate choice depends on whether the array is static or dynamic, whether the merge operation is invertible or idempotent, and whether updates are confined to single points or must span ranges. The Segment Tree is therefore a flexible general-purpose option when both range queries and updates must be supported efficiently.



Reference Pseudocode

The following compact templates summarize the core routines described throughout the report, including the lazy-propagation extension of Chapter 8. They intentionally expose the merge operation and the identity element so that the same structure can be adapted to sums, minima, maxima, GCD, or another associative aggregate (Chapter 9).

Listing A.1. Core build, point-update, and range-query routines (no lazy propagation).

```
build(node, lo, hi):
    if lo == hi:
        tree[node] = A[lo]
        return
    mid = (lo + hi) // 2
    build(2*node, lo, mid)
    build(2*node+1, mid+1, hi)
    tree[node] = merge(tree[2*node], tree[2*node+1])

update(node, lo, hi, pos, value):
    if lo == hi:
        tree[node] = value
        return
    mid = (lo + hi) // 2
    if pos <= mid:
        update(2*node, lo, mid, pos, value)
    else:
        update(2*node+1, mid+1, hi, pos, value)
    tree[node] = merge(tree[2*node], tree[2*node+1])

query(node, lo, hi, ql, qr):
    if qr < lo or hi < ql:
        return identity
    if ql <= lo and hi <= qr:
        return tree[node]
    mid = (lo + hi) // 2
    left = query(2*node, lo, mid, ql, qr)
    right = query(2*node+1, mid+1, hi, ql, qr)
    return merge(left, right)
```

Listing A.2. Lazy-propagation extension for range-add / range-sum (Chapter 8).

```
pushDown(node, lo, hi):
    if lazy[node] == 0:
        return
    mid = (lo + hi) // 2
    applyTag(2*node, lo, mid, lazy[node])
    applyTag(2*node+1, mid+1, hi, lazy[node])
    lazy[node] = 0

applyTag(node, lo, hi, val):
    tree[node] += val * (hi - lo + 1)
```

```
    lazy[node] += val

rangeUpdate(node, lo, hi, ul, ur, val):
    if ur < lo or hi < ul:
        return
    if ul <= lo and hi <= ur:
        applyTag(node, lo, hi, val)
        return
    pushDown(node, lo, hi)
    mid = (lo + hi) // 2
    rangeUpdate(2*node, lo, mid, ul, ur, val)
    rangeUpdate(2*node+1, mid+1, hi, ul, ur, val)
    tree[node] = tree[2*node] + tree[2*node+1]

lazyQuery(node, lo, hi, ql, qr):
    if qr < lo or hi < ql:
        return 0
    if ql <= lo and hi <= qr:
        return tree[node]
    pushDown(node, lo, hi)
    mid = (lo + hi) // 2
    left = lazyQuery(2*node, lo, mid, ql, qr)
    right = lazyQuery(2*node+1, mid+1, hi, ql, qr)
    return left + right
```

B

Summary of Complexity

Table B.1. Final complexity reference for every structure discussed in this report.

Structure	Update	Query	Extra Space
Naïve array	$O(1)$	$O(n)$	$O(n)$
Prefix sum	$O(n)$	$O(1)$	$O(n)$
Sqrt decomposition	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(n)$
Sparse table	unsupported	$O(1)$	$O(n \log n)$
Fenwick tree	$O(\log n)$	$O(\log n)$	$O(n)$
Segment tree (point update)	$O(\log n)$	$O(\log n)$	$O(n)$ tree storage
Segment tree (lazy, range update)	$O(\log n)$	$O(\log n)$	$O(n)$ tree + lazy storage

Bibliography

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [2] A. Laaksonen, *Competitive Programmer's Handbook*, 2018.
- [3] cp-algorithms, "Segment Tree," *Algorithms for Competitive Programming*. https://cp-algorithms.com/data_structures/segment_tree.html
- [4] P. M. Fenwick, "A New Data Structure for Cumulative Frequency Tables," *Software: Practice and Experience*, vol. 24, no. 3, pp. 327–336, 1994.
- [5] cp-algorithms, "Fenwick Tree," *Algorithms for Competitive Programming*. https://cp-algorithms.com/data_structures/fenwick.html
- [6] cp-algorithms, "Sparse Table," *Algorithms for Competitive Programming*. https://cp-algorithms.com/data_structures/sparse-table.html